UNM STUDENT SECTION

Code to Cloud: The Full-Stack IoT Portfolio Workshop

Beginner Web Workshop

Chapter 1 HTML

Chapter 2 CSS

Chapter 3 JavaScript

SA Disclaimer Form

UNM STUDENT SECTION

EPF00005644

UNM STUDENT SECTION

What You'll Build

A live web dashboard that talks to an ESP32 microcontroller via Firebase cloud.

1 HTML

Structure

Define your page layout with semantic tags: headings, divs, inputs, buttons, and text spans.

`index.html`

2 CSS

Styling

Make it look great — colors, spacing, cards, responsive layout, and flexbox.

`styles.css`

3 JavaScript

Logic + Firebase

Connect to Firebase Realtime Database, listen for data changes, and send commands.

`app.js`

UNM STUDENT SECTION

What is HTML?

HTML (HyperText Markup Language) is the skeleton of every webpage. It defines what content exists and how it is structured — not how it looks.

`<h1> … <h6>`
Headings — page titles and section names

`<input>`
Form control — slider, text box, etc.

`<p>`
Paragraph — blocks of text content

`<button>`
Clickable action trigger

`<div>`
Container — groups related elements

`<script>`
Loads JavaScript logic

`<span>`
Inline container — highlights a value

`<link>`
Links external CSS stylesheet

A webpage's anatomy:

```
<html>

  <head>
    <!-- tells browser: text encoding & mobile scaling -->
    <meta>

    <!-- text shown in the browser tab -->
    <title>

    <!-- loads external CSS stylesheet -->
    <link>
  </head>

  <body>
    <!-- heading -->
    <h1>
    <!-- paragraph -->
    <p>
    <!-- container / group -->
    <div>
      <!-- sub-heading -->
      <h2>
      <!-- paragraph -->
      <p>
      <!-- form control (slider, text box) -->
      <input>
      <!-- clickable action trigger -->
      <button>
    </div>

    <!-- loads JavaScript logic -->
    <script>

  </body>

</html>
```

HTML — Head & Servo Card

Part 1 · Page Head

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width,
initial-scale=1.0" />
  <title>ESP32 Dashboard</title>
  <link rel="stylesheet" href="styles.css" />
</head>

<body>
  <div style="text-align: center; margin-bottom: 50px;">
  <img
    src="Images/IEM 2021 Logo Blue Edition.png"
    alt="IEM 2021 Logo"
    style="max-width: 220px; width: 100%; margin-bottom:
20px;"
  />
  <h1 style="margin: 0;">ESP32 Firebase Dashboard</h1>
</div>

  <!-- Shows Firebase connection status -->
  <p class="status" id="status">Connecting...</p>
```

Part 2 · Servo Control Card

```html
<!-- ========== SERVO CONTROL CARD ========== -->
<div class="card">

  <h2>Servo Control</h2>

  <!-- Shows current slider angle -->
  <p>Angle: <span id="angleValue">90</span>°</p>

  <!-- Slider used to control servo -->
  <input id="angleSlider" type="range"
   min="0" max="180" value="90">

  <!-- Shows the value currently stored in
   Firebase -->
  <p>ESP32 angle: <span    id="servoAngle">90
  </span>° </p>
```

*id attributes let JavaScript find and update these elements in real time

HTML — IR Sensor & Messages Card

Part 3 · IR Sensor Card

```html
<!-- ========== IR SENSOR CARD ========== -->
<div class="card">

  <h2>IR Sensor</h2>

  <!-- Shows if object detected -->
  <p>Detected: <span id="irDetected">
   --</span></p>

  <!-- Shows raw sensor value -->
  <p>Raw: <span id="irRaw">--</span></p>

</div>
```

Element → Firebase path mapping:

#angleSlider	→	servo/msgTitle
#irDetected	→	sensor/obstacle
#irRaw	→	sensor/obstacle
#messages	→	serial/message

Part 4 · Message Terminal

```html
<!-- ========== MESSAGE TERMINAL CARD ========== -->
<div class="card">

  <h2>Messages</h2>

  <!-- Message display area -->
  <div id="messages">No messages yet.</div>

  <br>

  <!-- Message input row -->
  <div class="row">

    <input id="msgInput" type="text" placeholder=
    "Type a message">

    <button id="sendBtn">Send</button>

  </div>

</div>
```

What is CSS?

CSS (Cascading Style Sheets) controls how HTML looks. Without CSS, every page looks identical — plain black text on white.

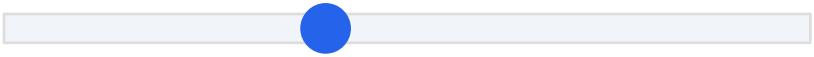
Anatomy of a CSS Rule

```
.card {                ← selector
  background: #fff;    ← property: value
  border-radius: 10px;
  padding: 18px;
}
```

Visual Result

Servo Control

Angle:



card { border-radius: 10px; padding: 18px; }

▦ Box Model

margin · border · padding · content

◈ Colors

background, color, border-color — hex or named

⇔ Flexbox

display:flex · gap · justify-content · align-items

CSS — Full Stylesheet

```
/* ========== GLOBAL STYLES ========== */
body {
  font-family: Arial, sans-serif;
  max-width: 700px;
  margin: 30px auto;
  padding: 20px;
  background: #f4f4f4;
  color: #222;
}

/* ========== CARD STYLES ========== */
.card {
  background: #fff;
  border: 1px solid #ddd;
  border-radius: 10px;
  padding: 18px;
  margin-bottom: 18px;
}

h1, h2 {
  margin-top: 0;
}

.status {
  font-weight: bold;
}

/* ========== FORM ELEMENTS ========== */
input[type="range"] {
  width: 100%;
}
```

▲ *Centers the dashboard and sets max width to 700 px*

```
/* ========== MESSAGE TERMINAL ========== */
#messages {
  height: 180px;
  overflow-y: auto;
  border: 1px solid #ccc;
  padding: 10px;
  background: #fafafa;
  white-space: pre-wrap;
}

/* ========== ROW LAYOUT (FLEXBOX) ========== */
.row {
  display: flex;
  gap: 10px;
}

input[type="text"] {
  flex: 1;
  padding: 8px;
}

button {
  padding: 8px 14px;
  cursor: pointer;
}
```

▲ *flex makes the input grow and button stay fixed*

What is JavaScript?

JavaScript is the brain of the webpage. It reads inputs, calls Firebase, and updates the display — all without reloading the page.

How It All Connects

Import Libraries

Load Firebase SDK modules at the top of app.js

Initialize App

Call initializeApp(config) with your project credentials

Get DOM Elements

Use getElementById to grab HTML controls

Listen & Update

onValue() watches Firebase and updates the page instantly

Write to Firebase

set(ref(db, path), value) pushes a new value to the cloud

JavaScript — Firebase Setup

Parts 1 & 2 · Import + Initialize Firebase

```javascript
/* ============================================
  PART 1: IMPORT FIREBASE LIBRARIES
============================================ */

import { initializeApp } from "https://www.gstatic.com/firebasejs/10.12.0/firebase-app.js";
import { getAuth, signInAnonymously, onAuthStateChanged }
from "https://www.gstatic.com/firebasejs/10.12.0/firebase-auth.js";

import { getDatabase, ref, onValue, set }
from "https://www.gstatic.com/firebasejs/10.12.0/firebase-database.js";


/* ============================================
  PART 2: FIREBASE CONFIGURATION
  REPLACE WITH YOUR OWN PROJECT SETTINGS
============================================ */

const firebaseConfig = {
  apiKey: "AIzaSyBlxLdiakI8Zv5gtRWtAcfwxTcW-XtGbTw",
  authDomain: "sivi-arduino-database.firebaseapp.com",
  databaseURL: "https://sivi-arduino-database-default-rtdb.asia-
southeast1.firebasedatabase.app",
  projectId: "sivi-arduino-database",
  storageBucket: "sivi-arduino-database.firebasestorage.app",
  messagingSenderId: "854752663108",
  appId: "1:854752663108:web:b80bad680b96bb63478e11",
  measurementId: "G-C1XE1J4ZJR"
};
/* ============================================
  INITIALIZE FIREBASE
============================================ */
const app = initializeApp(firebaseConfig);
const auth = getAuth(app);
const db = getDatabase(app);
```

Firebase Console Setup

1. **Create Project**

   console.firebase.google.com → Add Project

2. **Realtime Database**

   Build → Realtime DB → Start in Test Mode

3. **Authentication**

   Build → Auth → Enable Anonymous Sign-In

4. **Copy Config**

   Settings → Your Apps → Web → copy block

JavaScript — DOM Elements + Event Listeners

Part 3 · Get HTML Elements

```javascript
/* ==========================================
  PART 3: GET HTML ELEMENTS
========================================== */

const statusEl    = document.getElementById("status");
const angleSlider = document.getElementById("angleSlider");
const angleValue  = document.getElementById("angleValue");
const servoAngle  = document.getElementById("servoAngle");
const irDetected  = document.getElementById("irDetected");
const irRaw       = document.getElementById("irRaw");
const messages    = document.getElementById("messages");
const msgInput    = document.getElementById("msgInput");
const sendBtn     = document.getElementById("sendBtn");

let lastEspMsg = "";
```

Part 5 & 6 · Slider Event + Button Listener

```javascript
/* ==========================================
  PART 5: SERVO SLIDER CONTROL Listener
========================================== */

angleSlider.addEventListener("input", () => {
  const angle = Number(angleSlider.value);
  angleValue.textContent = angle;
  set(ref(db, "servo/msgTitle"), angle);
});

/* ==========================================
  PART 6: Send button Listener
========================================== */
// BUTTON EVENTS
sendBtn.addEventListener("click", sendWebMessage);
msgInput.addEventListener("keydown", (e) => {
  if (e.key === "Enter") {
    sendWebMessage();
  }
});
```

Part 4 · Serial Display

```javascript
/* ==========================================
PART 4   : Serial Display
========================================== */

// FUNCTION: DISPLAY MESSAGE
function addMessage(text, source) {
  if (messages.textContent === "No messages yet.") {
    messages.textContent = "";
  }
  messages.textContent += `[${source}] ${text}\n`;
  messages.scrollTop = messages.scrollHeight;
}


// FUNCTION: SEND MESSAGE FROM WEB
function sendWebMessage() {
  const text = msgInput.value.trim();
  if (!text) return;
  addMessage(text, "Web");
  msgInput.value = "";
  set(ref(db, "serial/message"), {
    text: text,
    source: "web"
  });
}
```

JavaScript — Firebase Real-Time Listeners

```javascript
/* ==========================================
  PART 7: Listen for Changes from Firebase
========================================== */

// LOGIN TO FIREBASE
signInAnonymously(auth).catch(err => {
  console.error(err)
  statusEl.textContent = "Auth failed";
});

// AFTER LOGIN - ATTACH REAL-TIME LISTENERS
onAuthStateChanged(auth, (user) => {
  if (!user) {
    statusEl.textContent = "Connecting...";
    return;
  }
  statusEl.textContent = "Connected";

  /* ----- SERVO LISTENER ----- */
  onValue(ref(db, "servo/msgTitle"), (snap) => {
    const val = snap.val();
    if (val === null) return;
    angleSlider.value = val;
    angleValue.textContent = val;
    servoAngle.textContent = val;
  });

  /* ----- IR SENSOR LISTENER ----- */
  onValue(ref(db, "sensor/obstacle"), (snap) => {
    const val = snap.val();
    if (val === null) return;
    irRaw.textContent = val;
    irDetected.textContent = val === 0 ? "YES" : "NO";
  });

  /* ----- MESSAGE LISTENER ----- */
  onValue(ref(db, "serial/message"), (snap) => {
    const data = snap.val();
    if (!data || !data.text) return;
    if (data.source === "esp32" && data.text !== lastEspMsg) {
      lastEspMsg = data.text;
      addMessage(data.text, "ESP32");
    }
  });
});
```

Part 7 · Listen for Changes from Firebase

onValue()

Fires every time the Firebase path changes — no polling needed

snap.val()

Returns the current data at that path (null if nothing saved yet)

Anonymous Auth

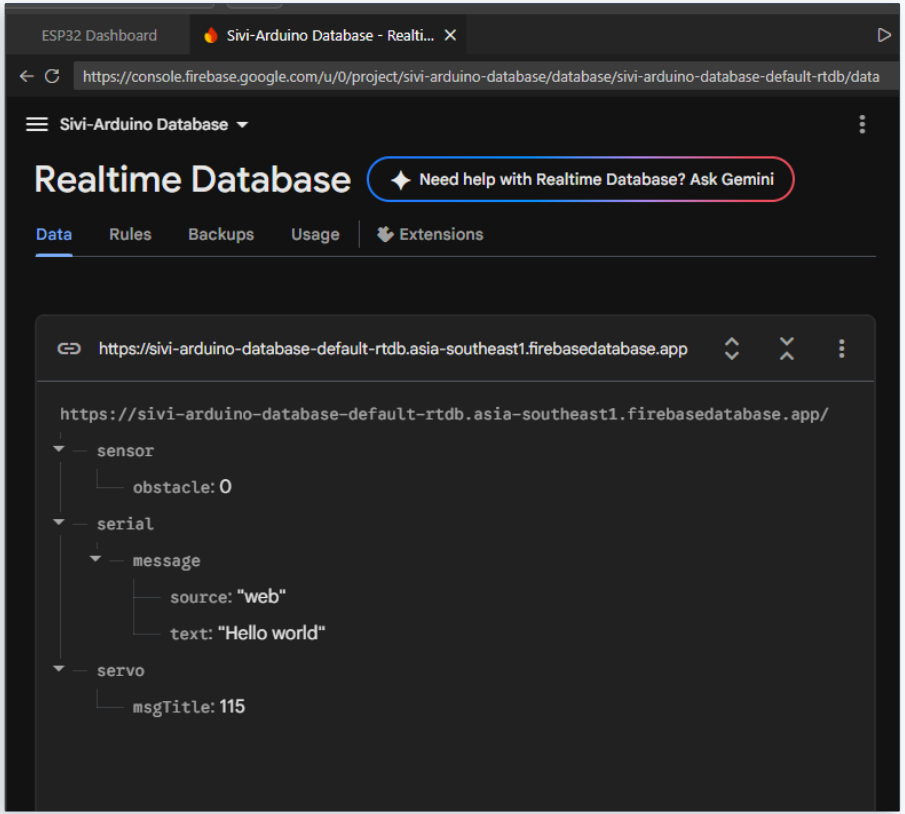
Simple, no login required — Firebase still knows each session

lastEspMsg guard

Prevents re-showing a message already displayed

Firebase Realtime Database Structure

What the database looks like



servo/msgTitle

Web → ESP32
Slider value (0–180) written by browser, read by ESP32 to move servo

sensor/obstacle

ESP32 → Web
Integer: 0 = object detected, 1 = clear (digitalRead behaviour)

serial/message

Bidirectional
Text + source — used for two-way chat between browser and ESP32

JavaScript Syntax — Variables, Functions & Events

UNM STUDENT SECTION

Key syntax used in app.js — explained line by line

const and let — Declaring Variables

```
const angleSlider = document.getElementById(...);
let lastEspMsg = "";
```

const — value never reassigned (use by default)
let — value can change later (e.g. lastEspMsg)
var — old style, avoid in modern JS

addEventListener — Reacting to Events

```
sendBtn.addEventListener("click", sendWebMessage);
msgInput.addEventListener("keydown", (e) => {
  if (e.key === "Enter") sendWebMessage();
});
```

element.addEventListener(event, callback)
Runs callback when event fires on element
Common events: click, input, keydown, change

Arrow Functions () => { }

```
angleSlider.addEventListener("input", () => {
  const angle = Number(angleSlider.value);
});
```

() => { } is shorthand for function() { }
(e) => { } receives an event object as parameter
Used everywhere callbacks are needed

Template Literals — Dynamic Strings

```
// Old way (string concatenation):
"[" + source + "] " + text + "\n"

// New way (template literal with backticks):
`[${source}] ${text}\n`
```

Use backtick ` not a quote " or '
${ } inserts any JS expression inside the string

JavaScript Syntax — Conditions, Callbacks & Comparisons

UNM STUDENT SECTION

More patterns from app.js — explained line by line

if and Guard Clauses

```
if (val === null) return;
if (!text) return;
if (!data || !data.text) return;
```

Guard clause: exit early if data is not ready
`!x` means "x is falsy" (null, 0, "", undefined)
`||` means "or" — either condition triggers exit

onValue Callback — snap.val()

```
onValue(ref(db, "servo/msgTitle"), (snap) => {
  const val = snap.val();
  if (val === null) return;
  angleSlider.value = val;
});
```

snap is a DataSnapshot object from Firebase
snap.val() extracts the raw JS value
Returns null if the path has no data yet

Ternary Operator — Inline if/else

```
irDetected.textContent = val === 0 ? "YES" : "NO";

condition ? valueIfTrue : valueIfFalse
```

IR sensor: 0 = object detected, 1 = clear
Shorter than writing a full if/else block

=== Strict Equality

```
if (val === null) return;        // exact
if (data.source === "esp32") ...  // exact
if (val === 0) ...               // exact
```

=== checks value AND type (always use this)
== only checks value — can cause surprises
!== is "not equal" (strict)

Workshop Summary

You now have a fully working real-time IoT dashboard. Here's what each file does:

3 files · 3 technologies · 1 live dashboard

index.html

HTML — Structure

- Defines the 3 cards: Servo, IR Sensor, Messages
- Uses id attributes so JS can find each element
- Loads styles.css and app.js

styles.css

CSS — Styling

- Centers dashboard at 700 px max-width
- Styles cards with border-radius + shadow
- Uses flexbox for the message input row

app.js

JavaScript — Logic + Firebase

- Imports Firebase SDK (app, auth, database)
- Writes slider value → servo/msgTitle on every move
- Listens to sensor/obstacle and serial/message with onValue()

Questions?

Raise your hand or ask in chat — happy to help!

Useful links:

Feedback Form

UNM STUDENT SECTION